

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

소스 취약성 식별 AI 모델 프로토타입:
AI Model Prototype for Source Code Vulnerability
Identification

본 문서는 “전기자동차 충전기 보안위협 대응 기술 개발” 과제를 수행하며 작성된
결과물임

2025. 12



	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

Copyright © 2025 KSIGN Co., Ltd. All rights reserved.

본 문서는 (주)케이사인에서 작성된 문서로 저작권법에 의해서 보호되고 있습니다. 따라서 본 기술문서의 저작권은 (주)케이사인에 있으며, 본사의 허가 없이 상기 상표를 무단 복제 혹은 사용하거나, 본 매뉴얼의 일부 또는 전체를 무단 복제, 전제 하실 수 없습니다.

경기도 과천시 과천대로 7나길 25, 13, 14층 (갈현동, 케이사인)

TEL : 02-564-0182 FAX : 02-564-1627

<http://www.ksign.com>

(주)케이사인

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

<목 차>

1	개요	8
1.1	목적	8
1.2	프로그램 구성 및 설계	8
2	Code Property Graph(CPG) 자동 생성 기술	10
2.1	Joern CPG	10
2.2	가상 컨테이너	12
2.3	병렬 처리 및 자동화	12
3	Template 변환	14
3.1	개요 및 설계	14
3.2	기능 및 컴포넌트 구성	14
3.3	실행 흐름	15
3.4	설정	17
4	AST 추출 모듈	19
4.1	개요	19
4.2	실행 흐름	19
4.3	컴포넌트 책임	21
4.4	데이터 변환	22
4.5	노드 및 엣지 타입	23
4.6	호출 의미론 및 플래그	23
5	GNN을 이용한 AST 기반의 취약점 탐지	25
5.1	개요	25
5.2	실행 흐름	26
5.3	컴포넌트 책임	27
5.4	데이터 변환	28
5.5	모델 아키텍처	28
5.6	설정	29
5.7	분석 파이프라인 통합	29
6	실험 및 성능 평가	30
6.1	실험 환경	30

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

6.2	성능 지표	31
6.3	성능 분석	32
6.4	오분류 분석	32
6.5	결론	33

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

<그림 목 차>

1	프로그램 전체 흐름 구성도	9
2	Docker 컨테이너 생성 스크립트	13
3	Template 변환 파이프라인 오케스트레이션	15
4	CPG 파싱 및 AST 추출 로직	15
5	노드 변환 디스패처	16
6	연산자 노드 변환 로직	16
7	제어 구조 정규화 로직	17
8	변수 선언 타입 분석 로직	17
9	이진 연산자 매핑 설정	17
10	단항 연산자 매핑 설정	18
11	AST 추출 모듈 초기화 로직	20
12	메인 실행 로직	20
13	블록 처리 및 노드 연결 로직	21
14	If 문 처리 및 가드 엣지 생성	22
15	AST 추출 모듈의 입력 및 출력 JSON 예시	22
16	학습 파이프라인 메인 로직	26
17	평가 파이프라인 로직	27
18	GNN 모델 구조	29

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

<표 목 차>

1	CPG 병렬 처리 속도 비교	12
2	전체 성능 지표	31
3	Confusion Matrix	31
4	상세 성능 지표	31

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

< 약 어 정 의 / Abbreviations >

AST Abstract Syntax Tree

CPG Code Property Graph

GNN Graph Neural Network

IR Intermediate Representation

PyG PyTorch Geometric

CLI Command-Line Interface

GINESStack Graph Isomorphism Network with Edge features

CFG Control Flow Graph

PDG Program Dependence Graph

 ℓ-Security Leader	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

1. 개요

1.1. 목적

본 프로젝트는 C/C++ 소스 코드의 보안 취약점을 탐지하는 AI 모델 프로토타입을 개발하여, 기존 정적 분석 도구의 구조적 한계를 해결하는 것을 목표로 한다. 기존 정적 분석 도구들은 규칙 기반 탐지에 의존함으로써 높은 False Positive Rate와 문맥 인식의 부재를 보이며, 실제 개발 현장에서 상당한 검증 비용과 피로도를 유발한다. 본 연구는 이러한 문제를 해결하기 위해 코드의 구조적 문맥 정보를 활용하는 학습 기반 접근 방식을 도입한다.

구체적으로, 소스 코드로부터 Abstract Syntax Tree(AST, 이하 AST)를 추출하여 그래프 표현으로 변환하고, 이를 Graph Neural Network(GNN, 이하 GNN)를 통한 학습으로 취약점 패턴을 인식한다. 이는 코드의 구조적 관계를 그래프 형태로 모델링함으로써, 단순 패턴 일치가 아닌 문맥을 고려한 정확한 취약점 탐지를 가능하게 한다.

1.2. 프로그램 구성 및 설계

본 시스템은 크게 그래프 데이터 추출 단계와 GNN 기반 분류 단계로 구성된다. 첫째, 소스 코드로부터 AST 그래프를 추출하는 전처리 파이프라인을 거치고, 둘째, 추출된 AST 그래프를 GNN 모델에 입력하여 취약점 여부를 분류한다.

1.2.1. 데이터 전처리 파이프라인

소스 코드를 GNN 학습에 적합한 그래프 표현으로 변환하기 위해 다음과 같은 단계를 거친다.

- **CPG 생성:** Joern을 활용하여 C/C++ 소스 코드를 CPG 로 변환한다. CPG는 코드의 구문, 제어 흐름, 타입 정보 등을 통합적으로 표현하는 멀티 레이어 그래프 구조이다.
- **Template 변환:** CPG의 복잡한 구조를 정규화하여 템플릿 그래프로 변환한다. 이 과정에서 연산자 이름 정규화(<operator>.addition → +), 제어 구조 표준화, 타입 추론 등이 수행된다. 이렇게 정규화된 템플릿 그래프는 이후 단계를 통해 기계 학습에 직접 활용 가능한 AST 기반 그래프 표현으로 변환된다.
- **AST 추출:** 템플릿 그래프에서 AST 구조만을 추출하여 계층적 트리 형태로 재구성한다. AST 추출 모듈은 이를 평탄화된 문장 수준 그래프로 변환하며, Parent-Child, Sibling, Guard 세 가지 유형의 엣지를 생성하여 구조적 관계를 보존한다.
- **특징 벡터 생성:** 각 노드에 대해 노드 타입, 루프 컨텍스트, 가드 조건, 호출 의미 등 GNN 학습에 필요한 수치형 특징 벡터를 계산한다.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

1.2.2. GNN 기반 분류 파이프라인

추출된 AST 그래프를 입력으로 받아 취약점을 탐지하는 과정은 다음과 같다.

- **그래프 인코딩**: AST 그래프를 GINeStack 인코더에 통과시켜 노드별 임베딩 벡터를 생성한다. GINeStack 은 그래프의 구조적 특성과 엣지 속성을 동시에 고려하여 노드 표현을 학습한다.
- **그래프 수준 표현**: Global Mean Pooling을 통해 노드별 임베딩을 집계하여 전체 그래프를 하나의 고정 길이 벡터로 변환한다. 이 그래프 임베딩 벡터는 함수 전체의 구조적 패턴을 요약한다.
- **분류**: 그래프 임베딩 벡터를 선형 분류기에 입력하여 해당 함수가 Vulnerable 인지 Safe 인지를 이진 분류한다. 학습 단계에서는 레이블링된 데이터셋을 사용하여 모델 파라미터를 최적화하고, 추론 단계에서는 학습된 모델을 사용하여 새로운 함수의 취약점 여부를 판별한다.

전체 파이프라인은 Figure 1에 도식화되어 있다.

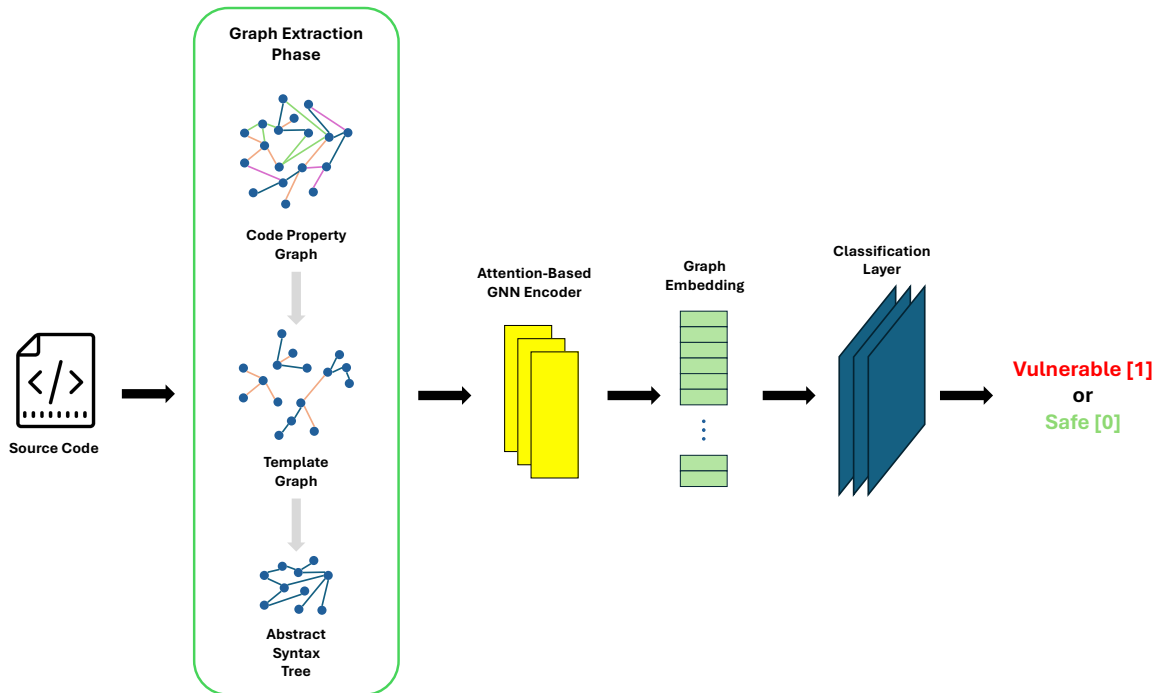


Figure 1: 프로그램 전체 흐름 구성도

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

2. Code Property Graph(CPG) 자동 생성 기술

본 문서 2절에서는 오픈소스 Joern 기반의 소스코드 그래프화에 대하여 기술한다.

2.1. Joern CPG

2.1.1. Joern

소스코드는 대체로 자연어 기반의 개발 친화적으로 비정형 데이터로 분류된다. 이러한 비정형 자연어는 정형화된 데이터를 처리하는 AI에게 가장 큰 병목으로, 이를 규격화하기 위한 시도들이 계속되고 있다. 이러한 접근들 중 하나로 오픈소스인 Joern이 존재한다. Joern은 Java 와 각 언어의 컴파일러 기반의 명령어 프로그램으로 소스코드를 다양한 형태의 **CPG**로 제공한다. Joern을 사용함으로써 가장 큰 장벽으로 Joern이 제공하는 그래프의 속성과 형태 그리고 다양한 Node와 Edge로 인해 분석에 어려움이 있다. Joern에서 제공하는 다양한 Node, Edge, Properties 중에 본 기술에서 사용된 일부는 2.1.2에 기술 되어있다. 이러한 복잡한 구조를 해결하기 위해 조금 더 사용자 친화적이고 AI가 처리할 수 있는 방향으로 전처리를 하게된다. 이렇게 전처리 후에 생성된 그래프를 본 문서에서는 **Template(템플릿)**이라고 규정한다.

2.1.2. CPG Specification

CPG 명세 1.1은 정적 분석을 위해 설계된 언어 중립적인 intermediate graph representation을 정의한다. CPG는 방향성 directed, 엣지 라벨 edge-labeled, 속성 attributed 멀티그래프 구조를 가지며, 코드의 각기 다른 측면을 모델링하는 여러 개의 레이어 Layer로 구성된다.

MetaData 레이어 이 레이어는 그래프의 생성 정보와 출처(provenance)를 기술한다.

- **노드:** META_DATA (언어, 명세 버전, 오버레이 정보 포함).
- **속성:** LANGUAGE, VERSION, OVERLAYS, ROOT, HASH.

FileSystem 레이어 그래프 노드를 실제 물리적 파일 시스템의 위치와 매핑한다.

- **노드:** FILE (소스 파일을 표현).
- **엣지:** SOURCE_FILE (노드를 원본 파일과 연결).
- **속성:** FILENAME, CODE, COLUMN_NUMBER, LINE_NUMBER.

Namespace 레이어 패키징 및 논리적 네임스페이스 구조를 다룬다.

- **노드:** NAMESPACE (논리적 네임스페이스), NAMESPACE_BLOCK (네임스페이스 내의 구체적인 코드 블록).
- **속성:** NAME, FULL_NAME.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

Method 레이어 파라미터와 반환 타입을 포함하여 메서드 선언을 모델링한다.

- **노드:** METHOD (함수 자체), METHOD_PARAMETER_IN, METHOD_PARAMETER_OUT, METHOD_RETURN.
- **속성:** SIGNATURE, IS_EXTERNAL, FULL_NAME, IS_VARIADIC.

Type 레이어 타입의 계층 구조, 선언 및 사용을 표현한다.

- **노드:** TYPE (구체적 인스턴스), TYPE_DECL (클래스/인터페이스 선언), MEMBER (필드), TYPE_ARGUMENT.
- **엣지:** BINDS_TO, INHERITS_FROM.
- **속성:** TYPE_FULL_NAME, INHERITS_FROM_TYPE_FULL_NAME.

AST 레이어 코드의 syntactic structure를 표현한다.

- **노드:** BLOCK, CALL, CONTROL_STRUCTURE, IDENTIFIER, LITERAL, RETURN, UNKNOWN, MODIFIER.
- **엣지:** AST (부모 노드와 자식 노드를 연결).
- **속성:** ORDER, CODE.

CallGraph 레이어 메서드 호출 및 dispatching 과정을 모델링한다.

- **노드:** CALL (호출 지점).
- **엣지:** CALL (발신 호출), RECEIVER, ARGUMENT.
- **속성:** METHOD_FULL_NAME, DISPATCH_TYPE, ARGUMENT_INDEX.

CFG 레이어 메서드 내에서의 실행 흐름(execution flow)을 모델링한다.

- **노드:** CFG_NODE (모든 제어 흐름 노드의 기본 클래스).
- **엣지:** CFG (실행 순서를 지시).

PDG 레이어 오염 분석 taint analysis 및 슬라이싱을 위해 데이터 및 제어 의존성을 모델링한다.

- **엣지:**
 - CDG (제어 의존성 그래프, Control Dependence Graph).
 - REACHING_DEF (데이터 의존성).
- **속성:** VARIABLE (전파되는 변수).

Binding 레이어 메서드 이름 및 시그니처를 타입 선언과 resolve한다.

- **노드:** BINDING (이름/시그니처를 메서드와 연결).
- **엣지:** BINDS (TypeDecl에서 Binding으로), REF (Binding에서 Method로).

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

보조 Auxiliary Layers 특정 분석 요구사항을 충족하기 위한 추가 레이어들이 존재한다.

- **Dominators:** DOMINATE 및 POST_DOMINATE 엣지를 포함한다.
- **Finding:** 분석 결과 저장을 위한 노드 (FINDING, KEY_VALUE_PAIR).
- **TagsAndLocation:** 임의의 노드 태깅을 위한 TAG 및 LOCATION 노드.

각 레이어별 상세 스키마, 노드 속성, 그리고 엣지 제약 조건에 대한 보다 구체적인 정보는 CPG 공식 웹사이트(<https://cpg.joern.io/>)에서 확인할 수 있다.

2.2. 가상 컨테이너

Joern은 활발히 개발·배포되는 도구로, 버전 변화에 따라 생성되는 CPG 스키마와 속성이 달라질 수 있다. 분석 파이프라인의 안정성과 재현성을 확보하기 위해, 본 연구에서는 Docker 기반 가상 컨테이너를 구축하여 Joern 실행 환경(버전, 의존 패키지, 설정 값 등)을 고정하였다. 이를 통해 Template 변환 및 후속 AST/그래프 변환 모듈이 항상 동일한 형태의 CPG를 입력으로 받도록 보장하며, 장기간에 걸친 반복 실험과 대규모 배치 전처리에서도 일관된 결과를 얻을 수 있다. 도커 컨테이너 설정은 다음 Figure 2와 같다.

해당 스크립트를 이용하여 도커 컨테이너를 생성한 후, 호스트 디렉토리를 컨테이너 내 `/workspace`와 같은 경로에 볼륨으로 마운트하여 동작시킨다. 이와 같은 구성은 CPG 생성 절차에서 호스트와 컨테이너 간의 파일을 안정적으로 공유하게 하고, 도커에 설치된 Joern을 통해 대규모 소스 코드를 일관된 환경에서 자동 변환할 수 있도록 지원한다.

2.3. 병렬 처리 및 자동화

하나의 소스 코드를 JSON 형태의 CPG로 생성하기 위해서는 Joern이 제공하는 두 가지 명령어를 순차적으로 실행해야 한다. `joern-parse`와 `joern-export`를 사용하여, 소스 코드를 컴파일과 동시에 바이너리 형태의 CPG로 변환하고 이를 JSON으로 익스포트하는 데에 각각 약 3~5초가 소요된다. 대량의 소스 코드를 전처리하는 경우, 직렬 방식으로는 약 1만 개의 소스 코드를 단순 CPG로 변환하는 데에만 8~13시간이 필요하다. 본 연구에서는 이러한 병목을 해소하기 위해 Python 기반 병렬 처리 로직을 도입하여, Joern 호출을 워커 단위로 분산 실행하였다. 그 결과, 약 21,000개의 데이터셋을 변환하는 데 필요한 전체 소요 시간을 8시간 수준으로 줄여, 파일당 평균 변환 시간을 **4.32초에서 1.37초로 단축**하고, 전체 처리 시간 기준으로 **68.3%** 수준의 성능 향상을 달성하였다.

	변환파일 수 (개)	소요 시간 (분)	파일당 소요시간 (초)	단축율 (%)
일반	10000	720	4.32	-68.3%
병렬	21000	480	1.37	

Table 1: CPG 병렬 처리 속도 비교

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

```

FROM ubuntu:22.04

ARG JOERN_VERSION=4.0.361
ENV DEBIAN_FRONTEND=noninteractive
ENV LANG=C.UTF-8 LC_ALL=C.UTF-8
ENV JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
ENV PATH="/usr/local/bin:${PATH}"
ENV JOERN_PORT=8080

RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        openjdk-17-jdk git unzip wget curl ca-certificates \
        xauth x11-apps locales tini && \
    rm -rf /var/lib/apt/lists/* && \
    update-alternatives --install /usr/bin/java java /usr/lib/jvm/java-17-
    openjdk-amd64/bin/java 1

RUN set -eux; \
    mkdir -p /opt/joern; \
    wget --progress=dot:giga -O /tmp/joern-cli.zip \
        "https://github.com/joernio/joern/releases/download/v${JOERN_VERSION}/
        joern-cli.zip"; \
    unzip -q /tmp/joern-cli.zip -d /opt/joern; \
    rm -f /tmp/joern-cli.zip; \
    ln -sf /opt/joern/joern-cli/joern /usr/local/bin/joern; \
    test -x /usr/local/bin/joern

RUN mkdir -p /workspace && chmod 777 /workspace
WORKDIR /workspace

RUN joern --help >/dev/null 2>&1 || true;

ENTRYPOINT ["/usr/bin/tini", "--"]

```

Figure 2: Docker 컨테이너 생성 스크립트

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

3. Template 변환

3.1. 개요 및 설계

Template 모듈은 정적 분석 파이프라인 내부에서 CPG 를 템플릿 분석에 적합한 형태로 변환하는 핵심 계층이다. 소스 코드는 일차적으로 CPG로 변환되며, 이 CPG는 타입 정보, 제어 흐름, 데이터 흐름, 호출 관계 등 매우 풍부한 의미 정보를 포함한다. 그러나 CPG 원본 데이터는 표현이 verbose 하고 그래프 구조가 복잡하며, 동일한 의미의 코드라도 표현 방식이 다를 수 있다는 한계가 있어 보안 패턴 매칭이나 템플릿 기반 분석에 직접 사용하기에는 부적합하다.

따라서 Template 모듈은 복잡한 CPG를 다음과 같이 체계적으로 가공하여, 후속 단계(패턴 매칭, 취약점 탐지, 코드 생성 등)에서 단일하고 일관된 IR 을 다룰 수 있도록 한다.

- **정규화:** `<operator>.addition`과 같은 내부 이름을 +와 같은 표준 연산자로 변환하고, 배열 선언 등 불일치한 표현을 일관된 템플릿 노드 타입으로 통합한다.
- **AST 재구성:** 그래프 엣지로 표현된 관계 중 AST 엣지만을 추출하여 직관적인 계층적 트리 구조를 재구성한다.
- **타입 추론 및 보강:** 암시적인 타입 정보를 추론하고, 원본 코드 메타데이터를 부착한다.
- **Flattening:** 필요 시 트리를 그래프 알고리즘에 적합한 노드/엣지 리스트 형태로 재가공한다.

설계 근거 및 전략:

- **컴포넌트 분리:** 추출, 변환, 후처리, 평탄화 단계를 명확히 분리하여 유지보수성과 테스트 용이성을 확보하였다.
- **재귀 처리:** AST의 중첩 구조를 직관적으로 처리하기 위해 재귀적 순회 방식을 채택하였다.
- **Graceful Degradation:** 비정상적인 CPG 입력(타입 정보 부족, 필수 속성 누락)에 대해서도 `<unknown>` 처리나 자식 노드 승격 등을 통해 분석을 지속한다. 단, 구조적 결함에 대해서는 명시적 예외를 발생시킨다.
- **설정 기반 관리:** 연산자 매핑이나 표준 라이브러리 목록을 코드가 아닌 별도 설정 파일로 관리하여 확장성을 높였다.

3.2. 기능 및 컴포넌트 구성

Template 모듈은 4단계의 파이프라인을 통해 CPG를 변환하며, 각 단계는 전담 컴포넌트가 수행한다.

1. **TemplateExtractor (추출):** CPG 그래프에서 AST 구조를 추출한다.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

- CPG JSON-LD 구조 검증 및 파싱 (@value 래퍼 해제).
- AST 엣지 필터링 및 nodeInfoMap, childrenMap 구성.
- 루트 노드(자식으로 등장하지 않는 노드) 식별 및 TreeNode 재귀적 생성.

2. **TemplateConverter (변환)**: 추출된 트리를 표준 템플릿 노드(TemplateNodes)로 변환한다.

- 노드 라벨(CALL, LOCAL, METHOD, CONTROL_STRUCTURE) 기반 디스패치.
- 연산자 이름 정규화 및 제어 구조 표준화.
- BinaryUnaryTypeWrapper를 이용한 Bottom-up 타입 추론 (실패 시 <unknown>).
- 표준 라이브러리 호출과 사용자 정의 호출 구분.

3.3. 실행 흐름

Template 변환은 buildTemplateArtifacts() 함수를 진입점으로 하여 순차적으로 실행된다. Endpoint는 용도에 따라 트리 기반의 templateResult를 활용하여 후속 분석을 수행한다.

```
function buildTemplateArtifacts(root: CPGRoot) {
  const extractor = new TemplateExtractor();
  const converter = new TemplateConverter();

  const template = extractor.getTemplateTree(root.export);
  const templateResult = converter.convertTree(template);

  return { template, templateResult };
}
```

Figure 3: Template 변환 파이프라인 오케스트레이션

3.3.1. Step 1: 트리 TemplateExtractor

CPG 데이터에서 @value 래퍼를 해제하고 AST 엣지만을 필터링하여 재귀적으로 구문 트리를 구성한다. 데이터 흐름이나 제어 흐름 엣지를 배제하여 순환 구조를 방지한다.

```
public getTemplateTree(cpg: unknown): TreeNode[] {
  if (!cpg || !("@value" in cpg)) return [];
  const inner = (cpg as ICPGRootExport)["@value"];
  const edges = inner.edges.filter(e => e.label === "AST");
  const nodes = inner.vertices;
  // Construct nodeInfoMap, identify root, create tree recursively
}
```

Figure 4: CPG 파싱 및 AST 추출 로직

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

3.3.2. Step 2: 노드 TemplateConverter

추출된 TreeNode를 표준화된 TemplateNodes로 변환한다. 각 노드의 라벨에 따라 적절한 핸들러로 Dispatch하며, 오류 발생 시 노드 식별 정보를 포함한 예외를 발생시킨다.

```
private dispatchConvert(node: TreeNode): TemplateNodes | undefined {
    switch (node.label) {
        case "CALL": return this.handleCall(node);
        case "CONTROL_STRUCTURE": return this.handleControlStructure(node);
        case "LOCAL": return this.handleLocal(node);
        case "METHOD": return this.handleMethod(node);
        default: return this.handleSkippedNodes(node);
    }
}
```

Figure 5: 노드 변환 디스패처

연산자 CALL <operator>.* 형태의 이름을 BinaryExpressionOperatorMap 등을 통해 표준 기호(+, ==)로 매핑하고 BinaryExpression 또는 UnaryExpression으로 변환한다.

```
private handleCallOperators(node: TreeNode) {
    if (BinaryExpressionOperatorMap[node.name]) {
        return {
            nodeType: TemplateNodeTypes.BinaryExpression,
            operator: BinaryExpressionOperatorMap[node.name],
            type: BinaryUnaryTypeWrapper(node),
            children: this.convertedChildren(node.children)
        };
    }
    // Handle UnaryExpression similarly
}
```

Figure 6: 연산자 노드 변환 로직

제어 Control Structure CONTROL_STRUCTURE_TYPE 속성을 확인하여 IF, WHILE 등으로 변환한다. IF 문의 경우 조건, True 블록, False 블록을 템플릿 포맷에 맞게 재배치하며, 자식 노드 수가 부족할 경우 예외를 발생시킨다.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

```
private handleControlStructure(node: TreeNode) {
  const type = node.properties.CONTROL_STRUCTURE_TYPE;
  type {
    case "IF":
      if (node.children.length < 2) throw new Error("Invalid IF structure");
      return { nodeType: TemplateNodeTypes.IfStatement,
        children: restructureIfChildren(node.children) };
    // Handle WHILE, FOR, SWITCH similarly
  }
}
```

Figure 7: 제어 구조 정규화 로직

변수 및 함수 처리 변수 선언(LOCAL)의 경우 TYPE_FULL_NAME을 분석하여 배열([], 포인터(*))를 구분한다. 함수(METHOD)는 파일 스코프 내의 항목만 처리하며, 본문 유무에 따라 Declaration과 Definition으로 나눈다.

```
// Variable declaration type analysis
if (typeFullName.includes("[") return { nodeType: TemplateNodeTypes.
  ArrayDeclaration };
if (typeFullName.includes("*") return { nodeType: TemplateNodeTypes.
  PointerDeclaration };
return { nodeType: TemplateNodeTypes.VariableDeclaration };
```

Figure 8: 변수 선언 타입 분석 로직

3.4. 설정

Template 모듈의 확장성을 위해 연산자 매핑, 표준 라이브러리 목록 등은 코드가 아닌 별도의 설정 파일로 관리된다.

- **BinaryExpression.ts**: CPG 이진 연산자를 표준 연산자로 매핑한다.

```
export const BinaryExpressionOperatorMap = {
  "<operator>.addition": "+",
  "<operator>.equals": "==",
  "<operator>.logicalAnd": "&&",
  // ... more operators
};
```

Figure 9: 이진 연산자 매핑 설정

- **UnaryExpression.ts**: 단항 연산자 매핑을 정의한다.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

```
export const UnaryExpressionOperatorMap = {
  "<operator>.postIncrement": "++",
  "<operator>.logicalNot": "!",
  "<operator>.indirection": "*",
  // ... more operators
};
```

Figure 10: 단항 연산자 매핑 설정

- **Predefined.ts:** stdin, NULL 등의 식별자에 대해 고정 타입을 부여한다.
- **StandardLibCall.ts:** 표준 라이브러리 및 보안 관련 함수 레지스트리를 관리하여 사용자 정의 호출과 구분한다.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

4. AST 추출 모듈

4.1. 개요

AST 추출 모듈은 계층적인 템플릿 JSON(함수 수준 AST)를 GNN 학습에 적합한 평탄화된 문장 수준 그래프 표현으로 변환하는 핵심 구성 요소이다. 이 모듈은 제어 흐름 관계, Guard 조건, 의미적 주석(Semantic Annotations)을 보존하면서 AST 정보를 추출한다.

4.1.1. 해결하고자 하는 문제

템플릿 JSON은 코드 함수의 계층적 AST 표현을 제공하지만, GNN 모델 학습을 위해서는 다음과 같은 요건이 충족되어야 한다.

- **구조 평탄화:** 깊게 중첩된 트리 구조보다 평탄한 노드 리스트가 GNN 처리에 효율적이다.
- **문장 수준 입도:** Expression 수준이 아닌 할당, 선언, 제어문 등 Statement 수준의 분석이 필요하다.
- **제어 흐름 보존:** 제어문(if, for, while, switch)과 그로 인해 실행되는 블록 간의 관계가 명시적으로 표현되어야 한다.
- **특징 추출:** 각 노드는 의미 정보(노드 타입, 루프 컨텍스트, 가드 제약, 호출 의미 등)를 포함한 수치형 특징을 가져야 한다.

AST 추출 모듈은 계층적 AST를 평탄화하고, 구조 및 제어 흐름을 보존하기 위해 세 가지 유형의 엣지(Parent-Child, Sibling, Guard)를 생성하며, 각 노드에 대해 GNN 입력에 적합한 특징 벡터를 계산함으로써 이러한 문제를 해결한다.

4.2. 실행 흐름

4.2.1. Initialization

AST 추출 모듈 인스턴스 생성 시, 입력된 템플릿 JSON을 순회하여 모든 노드를 ID로 인덱싱하고 결과 컨테이너를 초기화한다.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

```

def __init__(self, ast_json: Dict[str, Any]):
    self.ast = ast_json
    self.idmap: Dict[int, Dict[str, Any]] = {}
    self.parent: Dict[int, int] = {}

    # Recursively index all nodes
    def _idx(n, parent_id: Optional[int] = None):
        if isinstance(n, dict) and (nid := n.get('id')):
            self.idmap[nid] = n
            if parent_id: self.parent[nid] = parent_id
            for c in (n.get("children") or []):
                _idx(c, nid)
    _idx(self.ast)

    # Initialize result containers
    self.nodes, self.edges_pc, self.edges_sb, self.edges_ast_guard = [], [], [], []
    self.nodes.append({"sid": 0, "node_type": "FunctionEntry", "feat": {...}})

```

Figure 11: AST 추출 모듈 초기화 로직

초기화 단계에서는 원본 AST 노드 ID 매핑(idmap), 부모 노드 추적(parent), 그리고 평탄화된 그래프를 저장할 컨테이너(nodes, edges_*)를 준비한다. 또한, 그래프의 루트 역할을 하는 FunctionEntry 노드(sid=0)를 생성한다.

4.2.2. 메인 추출 프로세스

run() 메서드는 파라미터 선언 처리, 함수 본문 평탄화, 제어 노드 후처리를 순차적으로 수행한다.

```

def run(self) -> Dict[str, Any]:
    self._emit_param_statements_prologue()
    func_body = next((c for c in self.ast.get("children", []))
                     if c.get("nodeType") == "CompoundStatement"), None)
    if func_body:
        self._process_block(func_body, 0, {}, 0)
    self._postprocess_control_calls()
    return {"nodes": self.nodes, "edges_ast_pc": self.edges_pc,
            "edges_ast_sb": self.edges_sb, "edges_ast_guard": self.edges_ast_guard}

```

Figure 12: 메인 실행 로직

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

4.3. 컴포넌트 책임

4.3.1. Block Processing

`_process_block` 함수는 `CompoundStatement`를 재귀적으로 순회하며 문장 수준 노드를 생성하고 엣지를 연결하는 핵심 역할을 수행한다.

```
def _process_block(self, block_node, parent_sid, active_guard, in_loop):
    sb_prev, first_sid = None, None
    for ch in block_node.get("children", []):
        if ch.get("nodeType") == "CompoundStatement":
            child_first, child_last = self._process_block(ch, parent_sid,
dict(active_guard), in_loop)
            if sb_prev: self.edges_sb.append((sb_prev, child_first, 1))
            sb_prev = child_last
        else:
            sid = self._make_node(...)
            self.edges_pc.append((parent_sid, sid, 0))
            if sb_prev: self.edges_sb.append((sb_prev, sid, 1))
            sb_prev = sid
    return first_sid, sb_prev
```

Figure 13: 블록 처리 및 노드 연결 로직

이 함수는 중첩된 제어 구조를 재귀적으로 처리하면서 계층 구조(PC 엣지)와 실행 순서(SB 엣지)를 보존한다.

4.3.2. Node Creation

`_make_node` 함수는 평탄화된 노드를 생성하고 GNN 학습에 필요한 모든 수치형 특징을 계산한다.

1. **ID 할당**: 순차적인 `sid`를 할당한다.
2. **호출 의미 분석**: 함수 호출 노드에 대해 의미 카테고리 및 위험 플래그를 계산한다.
3. **버퍼 선언 분석**: 배열 선언에 대해 크기 및 상태 정보를 추출한다.
4. **컨텍스트 분석**: 루프 내부 여부, 가드 강도 등을 계산한다.
5. **특징 벡터 생성**: `feat` 딕셔너리를 구성하고 노드 객체를 생성한다.

4.3.3. Control Flow Handling

If, For, While, Switch 제어문은 Guard Edge를 통해 제어 노드와 피제어 블록을 연결한다.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

```

if t == "IfStatement":
    sid_if = self._make_node("IfStatement", cond_code, in_loop, 0, ...)
    if then_block:
        then_first, _ = self._process_block(then_block, sid_if, ..., in_loop)
        if then_first: _emit_guard(sid_if, then_first, 1, 0) # THEN branch
    if else_block:
        else_first, _ = self._process_block(else_block, sid_if, ..., in_loop)
        if else_first: _emit_guard(sid_if, else_first, 1, 1) # ELSE branch

```

Figure 14: If 문 처리 및 가드 엣지 생성

4.3.4. Guard Edge Emission

_emit_guard 함수는 제어 흐름의 종류와 분기를 나타내는 속성을 포함하여 가드 엣지를 생성한다.

- **guard_kind**: 1(If), 2(Loop), 4(Switch).
- **guard_branch**: If(0=Then, 1=Else), Loop(2), Switch(Case 값).

4.4. 데이터 변환

4.4.1. 입출력 구조

입력은 계층적인 템플릿 JSON이며, 출력은 노드 리스트와 엣지 리스트로 구성된 평탄화된 그래프이다.

Input JSON

```

{
  "nodeType": "
    FunctionDefinition",
  "children": [
    ...
  ]
}

```

Output JSON

```

{
  "nodes": [
    ...
  ],
  "edges_ast_pc": [
    ...
  ],
  "edges_ast_sb": [
    ...
  ],
  "edges_ast_guard": [
    ...
  ]
}

```

Figure 15: AST 추출 모듈의 입력 및 출력 JSON 예시

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

4.4.2. Node Features

각 노드의 feat 딕셔너리는 다음 정보를 포함한다.

- 기본 정보: node_type_id, train_mask.
- 컨텍스트: in_loop, is_loop, ctx_guard_strength.
- 버퍼 정보: is_buffer_decl, buffer_size_norm.
- 호출 의미: call_sem_cat_id, call_flag_danger_unbounded 등.

4.5. 노드 및 엣지 타입

4.5.1. 지원 노드 타입

NODE_TYPE_ID 매핑을 통해 정수형 ID로 변환된다.

- 선언: VariableDeclaration(3), ArrayDeclaration(4), PointerDeclaration(5).
- 제어: IfStatement(10), ForStatement(11), WhileStatement(12), SwitchStatement(14).
- 호출: StandardLibCall(30), UserDefinedCall(31).
- 기타: FunctionEntry(1), AssignmentExpression(6).

4.5.2. 엣지 타입

세 가지 엣지 패밀리를 통해 코드의 구조적 정보를 보존한다.

1. **Parent-PC**: 원본 AST의 계층 구조 표현 (엣지 타입 0).
2. **SB**: 문장 간의 실행 순서 표현 (엣지 타입 1).
3. **Guard**: 제어 흐름에 따른 조건부 실행 관계 표현 (엣지 타입 2).

4.6. 호출 의미론 및 플래그

4.6.1. 호출 카테고리

함수 호출은 취약점 분석을 위해 다음 카테고리로 분류된다.

- 메모리 조작: mem_alloc(1), mem_copy(2), mem_set(5).
- 입출력: ext_input(3), format_print(4).
- 기타: net_connect(6), parse_int(9/10).

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

4.6.2. 호출 플래그 계산

compute_call_flags 함수는 호출의 위험성을 판단하기 위한 상세 플래그를 계산한다.

- **Unbounded Call:** strcpy, gets 등 경계 검사가 없는 호출 식별.
- **Size Relationship:** 인자가 sizeof(dst)와 연관되어 있는지 분석.
- **Field Sensitivity:** 목적지가 구조체 필드인지 여부 확인.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

5. GNN을 이용한 AST 기반의 취약점 탐지

5.1. 개요

취약점 탐지 모듈은 정적 분석 파이프라인의 후반부에서 동작하는 그래프 신경망(GNN) 기반의 취약점 탐지 시스템이다. 이 모듈은 앞선 단계에서 생성된 코드의 AST 표현을 입력으로 받아, 해당 함수가 Vulnerable 인지 혹은 Safe 인지를 분류하도록 학습된 PyG 기반 AI 모델을 운용한다.

5.1.1. 해결하고자 하는 문제

기존의 정적 분석 도구는 잠재적 취약점을 일정 수준까지 식별할 수 있으나, 코드의 Semantics 와 Context를 충분히 이해하지 못해 다음과 같은 한계가 존재한다.

- **Context Sensitivity 부족:** 동일한 코드 패턴이라도 문맥에 따라 안전할 수도, 취약할 수도 있다.
- **False Positives 증가:** 런타임 제약 조건이나 검증 로직으로 인해 실제로는 안전한 코드임에도 취약점으로 탐지되는 경우가 빈번하다.
- **복잡한 패턴:** 취약점은 단일 구문이 아닌, 여러 코드 요소 간의 복잡한 상호작용으로 인해 발생하는 경우가 많다.

AI 모델은 레이블링된 데이터셋을 통해 GNN 모델을 학습시킴으로써 이러한 문제를 해결한다. GNN 아키텍처는 코드를 그래프 구조(구문을 위한 AST)로 표현하고, 노드 및 엣지 속성으로부터 Feature를 자동으로 학습하여 구문적 패턴을 포착하는 데 최적화되어 있다.

5.1.2. 기능 요약

취약점 탐지 모듈은 다음과 같은 기계 학습 파이프라인 전 과정을 제공한다.

1. **데이터 로딩:** 함수별 AST 그래프가 포함된 JSON 파일을 로드한다.
2. **그래프 구축:** JSON 데이터를 PyG의 Data 객체(노드 특징, 엣지 인덱스, 엣지 속성 포함)로 변환한다.
3. **모델 학습:** 학습 데이터를 사용하여 함수를 취약(1) 또는 안전(0)으로 분류하는 GNN 모델을 학습한다.
4. **모델 평가:** 테스트 데이터를 통해 모델의 정확도, Confusion Matrix, 클래스별 통계 등을 평가한다.
5. **설정 관리:** 재현성을 보장하기 위해 전체 학습 구성을 저장하고 관리한다.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

5.2. 실행 흐름

5.2.1. 학습 파이프라인

학습 프로세스는 `train()` 함수 호출 시 시작되며, 데이터 로딩부터 모델 저장까지의 과정을 수행한다.

```
def train(cfg: Optional[TrainConfig] = None) -> None:
    cfg = cfg or _parse_train_args()
    torch.manual_seed(cfg.seed)
    device = torch.device(cfg.device)

    # Load datasets and create data loaders
    train_dataset, test_dataset = load_and_split_datasets(cfg.data_path)
    train_dataloader = build_dataloader(train_dataset, cfg, collate_fn=collate)
    test_dataloader = build_dataloader(test_dataset, cfg, collate_fn=collate)

    # Initialize model
    dims = infer_dims_from_dataset(train_dataset, kinds=["ast"])
    model = select_model(cfg, dims["ast"][0], dims["ast"][1]).to(device)

    # Setup training
    optimizer = torch.optim.Adam(model.parameters(), lr=cfg.lr)
    criterion = nn.CrossEntropyLoss(weight=compute_class_weights(...))

    # Train and save
    train_model_from_dataset(cfg, train_dataloader, model, optimizer,
                             criterion, device)
    torch.save(model.state_dict(), weights_path)
    save_training_config(cfg, results_dir, model_info)
```

Figure 16: 학습 파이프라인 메인 로직

주요 단계별 상세 내용은 다음과 같다.

- **구성 파싱:** CLI 인자를 통해 `TrainConfig` 객체를 생성하여 유연한 실험 설정을 지원한다.
- **데이터셋 로딩:** `GenericJsonDataset`을 통해 JSON 파일을 PyG 객체로 변환하고, 파일명 키워드 등을 기반으로 레이블을 추론한다.
- **데이터로더 생성:** `collate` 함수를 사용하여 AST 그래프 배치를 구성한다.
- **차원 추론:** 데이터셋을 검사하여 가변적인 노드/엣지 속성의 차원을 자동으로 계산한다.
- **모델 선택:** AST 그래프를 처리하는 GNN 모델을 인스턴스화한다.
- **학습 및 저장:** 에포크별 학습, 체크포인트 저장, 손실 기록, 최종 모델 및 설정 파일 저장을 수행한다.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

5.2.2. 평가 파이프라인

평가 프로세스는 저장된 설정을 로드하여 학습 환경을 복원한 후 수행된다.

```
def evaluate() -> None:
    config = load_training_config(results_dir)
    cfg = TrainConfig(**config["training_config"])
    train_dataset, test_dataset = reconstruct_datasets(cfg)
    model = load_model_robust(checkpoint_path, device)
    summary = evaluate_model(model, eval_dataloader, device, cfg.mode)
    save_evaluation_results(summary, "evaluation.json")
```

Figure 17: 평가 파이프라인 로직

5.3. 컴포넌트 책임

5.3.1. GenericJsonDataset

JSON 파일을 로드하고 PyG 그래프 객체로 변환하는 역할을 담당한다. Pydantic 모델을 사용하여 JSON 스키마를 검증하며, juliet_json_to_sample 변환기를 통해 AST 섹션을 추출하고 그래프를 구축한다.

5.3.2. Graph Building Functions

JSON 구조의 노드와 엣지를 PyG 텐서로 변환한다.

- **노드 특징:** JSON의 feat 필드나 노드 자체를 플랫폼하게 펼쳐 특징 행렬 x를 구성한다.
- **엣지 구성:** edges_ast 패밀리를 식별하고, 원-핫 인코딩된 패밀리 정보와 엣지 속성을 결합하여 edge_attr 및 edge_index를 생성한다.
- **차원 조화:** _harmonize_graph_dims 함수를 통해 배치 내 모든 그래프가 호환 가능한 차원을 갖도록 패딩한다.

5.3.3. Collate Function

AST 그래프와 가변 메타데이터를 포함하는 샘플들을 하나의 배치로 결합한다. PyG의 Batch.from_data_list를 사용하여 효율적인 병렬 처리가 가능한 연결된 그래프(Disjoint Union) 형태를 생성한다.

5.3.4. Model Selection & Training

설정된 모드에 따라 적절한 모델 아키텍처를 선택하고 학습 루프를 제어한다. AST 그래프를 입력으로 받아 취약점 여부를 분류한다.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

5.3.5. Evaluation Functions

`evaluate_model` 및 `analyze_sample_with_model` 함수를 통해 추론을 수행하고, 정확도, Confidence, 오분류 샘플 등의 상세 분석 결과를 생성한다.

5.4. 데이터 변환

5.4.1. JSON → PyG Graph

입력 JSON은 AST 노드 및 엣지 정보를 포함하며, 다음과 같은 과정을 거쳐 PyG Data 객체로 변환된다.

- 노드 특징 추출:** 모든 노드의 고유한 속성 키를 수집하여 특징 행렬을 구성한다 (누락된 속성은 0으로 패딩).
- 엣지 수집 및 특징화:** 엣지 패밀리(타입)를 식별하여 원-핫 벡터를 생성하고, 추가 속성(Guard kind 등)과 결합한다.
- 텐서 생성:** 최종적으로 `x`, `edge_index`, `edge_attr` 텐서와 레이블 `y`, 메타데이터를 포함하는 객체를 생성한다.

5.4.2. Graph Batching

배치 처리 시 차원 조화(Dimension Harmonization) 과정을 통해 서로 다른 특징 집합을 가진 그래프들을 정렬하고, 이를 하나의 거대한 그래프 객체(Batch)로 병합하여 GPU 연산 효율성을 극대화한다.

5.5. 모델 아키텍처

5.5.1. GNN Model Architecture

본 시스템은 AST 그래프를 처리하는 GNN 모델을 사용한다. GINESStack 을 사용하여 그래프를 인코딩하고, 글로벌 풀링 후 선형 분류기를 통해 예측을 수행한다.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

```

class ASTGNNModel(nn.Module):
    def __init__(self, ast_in, ast_edge_dim, cfg):
        self.encoder = GINStack(cfg.num_layers, ast_in, cfg.hidden_dim,
                                ast_edge_dim)
        self.pool = global_mean_pool
        self.classifier = nn.Linear(cfg.hidden_dim, cfg.out_classes)

    def forward(self, ast_data):
        ast_repr = self.encoder(ast_data.x, ast_data.edge_index, ast_data.
                                edge_attr)
        return self.classifier(self.pool(ast_repr, ast_data.batch))

```

Figure 18: GNN 모델 구조

- **Encoder:** GINStack을 사용하여 AST 그래프의 구조적 특성을 학습한다.
- **Pooling:** 글로벌 평균 풀링을 통해 그래프 수준의 표현을 생성한다.
- **Classifier:** 학습된 그래프 표현을 바탕으로 취약점 여부를 분류한다.

5.6. 설정

TrainConfig 클래스는 학습에 필요한 모든 하이퍼파라미터와 데이터 경로를 정의한다.

- **기본 설정:** 에포크 수, 학습률, 배치 크기, 디바이스, GNN 레이어 수, 히든 차원 등.
- **데이터 경로:** JSON 파일 경로 및 레이블 추론 규칙(label_key)을 정의한다.
- **레이블 추론:** 명시적 레이블 필드, 파일명 키워드 매칭("bad", "patched"), 휴리스틱 순으로 레이블을 결정한다.

학습 시 training_config.json 파일이 생성되어 실험의 재현성을 보장하며, 모델 로딩 시 수동 설정 없이도 아키텍처를 복원할 수 있게 한다.

5.7. 분석 파이프라인 통합

취약점 탐지 모듈은 전체 정적 분석 파이프라인의 최종 단계로 통합된다.

1. **CPG 생성:** Joern을 이용해 소스 코드를 CPG로 변환.
2. **Template 생성:** CPG를 정규화된 템플릿 노드로 변환.
3. **Graph 추출:** 템플릿에서 AST 그래프 추출 및 JSON 익스포트.
4. **GNN 분석:** AI 모델이 JSON을 입력받아 취약점 여부를 판별.

CLI 명령어 `uv run train` 및 `uv run evaluate`를 통해 독립적으로 실행 가능하며, 학습된 모델은 배치 예측이나 정적 분석 결과의 필터링 용도로 배포되어 활용된다.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

6. 실험 및 성능 평가

본 절에서는 제안된 AST 기반 GNN 모델의 취약점 탐지 성능을 실제 데이터셋을 통해 평가한 결과를 제시한다.

6.1. 실험 환경

6.1.1. 데이터셋

실험에는 NIST Juliet Test Suite v1.3의 C/C++ 데이터셋을 활용하였다. Juliet Test Suite는 다양한 소프트웨어 취약점 유형을 체계적으로 포함하는 표준 벤치마크 데이터셋으로, 본 연구에서는 다음 세 가지 데이터 소스를 사용하였다.

- **CWE-121** (data/CWE121_Stack-Based_Buffer_Overflow): 총 18,636개 샘플, Label 1 (취약) 16,156개, Label 0 (안전) 2,480개
- **CWE-122** (data/CWE122_Heap-Based_Buffer_Overflow): 총 23,488개 샘플, Label 1 (취약) 19,332개, Label 0 (안전) 4,156개
- **Real-world Code** (data/real_code): 총 51개 샘플, Label 1 (취약) 16개, Label 0 (안전) 35개

각 Juliet 폴더에서는 파일명에 bad 키워드를 포함하는 샘플을 Label 1 (취약)로, 그렇지 않은 샘플을 Label 0 (안전)으로 라벨링하였다. data/real_code 폴더에서는 파일명이 patched 인 샘플을 Label 0 (안전)으로, 그렇지 않은 샘플을 Label 1 (취약)으로 정의하였다. 전체 기준으로는 총 42,175개 샘플 중 취약(Label 1)이 35,504개(84.18%), 안전(Label 0)이 6,671개(15.82%)로, 취약 클래스가 우세한 불균형 데이터셋이다. 모델 평가는 이 전체 데이터셋으로부터 무작위로 추출한 테스트 세트 4,218개 샘플에 대해 수행되었다.

6.1.2. 실험 설정

모델 학습 및 평가는 다음과 같은 설정으로 수행되었다.

- **데이터 분할**: 전체 데이터셋을 학습 세트와 테스트 세트로 분할 (비율은 데이터셋 특성에 따라 결정)
- **모델 아키텍처**: GINeStack 기반 인코더, 히든 차원 128, 레이어 수 3
- **학습 파라미터**: 학습률 0.001, 배치 크기 32, 에포크 수 50
- **하드웨어**: NVIDIA GPU를 활용한 병렬 학습

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

6.2. 성능 지표

6.2.1. 전체 성능

테스트 세트 4,218개 샘플에 대한 평가 결과는 다음과 같다.

지표	값	비율
총 샘플 수	4,218	100.00%
Accuracy	3,996	94.74%

Table 2: 전체 성능 지표

6.2.2. Confusion Matrix

이진 분류 결과를 Confusion Matrix로 정리하면 Table 3와 같다.

실제	예측		합계
	취약 (1)	안전 (0)	
취약 (1)	3,441 (TP)	129 (FN)	3,570
안전 (0)	93 (FP)	555 (TN)	648
합계	3,534	684	4,218

Table 3: Confusion Matrix

6.2.3. 상세 성능 지표

혼동 행렬을 바탕으로 계산된 주요 성능 지표는 Table 4와 같다.

지표	값
Accuracy	94.74%
Recall / TPR	96.39%
Precision	97.37%
F1-Score	96.88%
FPR	14.35%
Specificity	85.65%

Table 4: 상세 성능 지표

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

6.3. 성능 분석

6.3.1. 높은 재현율의 의미

Recall 96.39%는 실제 취약점 중 96.39%를 정확히 탐지함을 의미한다. 이는 보안 도구로서 가장 중요한 지표 중 하나로, False Negative를 최소화하여 실제 취약점을 놓치지 않는 것이 핵심이기 때문이다. 본 모델은 3,570개의 실제 취약점 중 3,441개를 정확히 탐지하여, 단 129개(3.61%)만을 놓쳤다.

6.3.2. 오탐율 분석

False Positive Rate 14.35%는 정상 코드 648개 중 93개를 잘못 취약점으로 분류한 것이다. 이는 기존 정적 분석 도구의 전형적인 False Positive Rate(20-40%) 대비 경쟁력 있는 수치이나, 여전히 개선의 여지가 있다. False Positive를 감소시키기 위해서는 다음과 같은 방향의 개선이 필요하다.

- **데이터 보강:** 정상 코드 패턴의 다양성을 높인 학습 데이터 추가
- **특징 개선:** 문맥 정보를 더 정교하게 반영하는 노드 특징 벡터 설계
- **양상불 기법:** 여러 모델의 예측을 결합하여 오탐 감소

6.3.3. 클래스 불균형 대응

전체 42,175개 샘플을 기준으로 할 때, 데이터셋은 취약(Label 1) 35,504개(84.18%), 안전(Label 0) 6,671개(15.82%)로 약 5.3:1의 심각한 클래스 불균형을 보인다. 이와 같은 불균형 상황에서도 무작위로 추출된 테스트 세트 4,218개 샘플에 대해 94.74%의 높은 정확도를 달성한 것은 다음과 같은 요인에 기인한다.

- **가중치 조정:** 학습 시 Class Weight를 적용하여 소수 클래스(안전)에 더 높은 학습 가중치 부여
- **특징 품질:** AST 구조 정보가 취약점과 정상 코드를 효과적으로 구분할 수 있는 특징을 제공
- **모델 용량:** GINestack의 충분한 표현력이 복잡한 패턴 학습을 가능하게 함

6.4. 오분류 분석

6.4.1. 높은 확신도의 오분류

모델 평가 결과, 일부 샘플에 대해서는 높은 Confidence를 가지고 오분류하는 사례가 발견되었다. 예를 들어, CWE122_..._rand_12_bad.json 파일의 경우 모델이 0.99의 높은 확신도로 취약점으로 분류했으나 실제로는 정상 코드였다.

이러한 현상은 다음과 같은 원인으로 분석된다.

	소스 취약성 식별 AI 모델 프로토타입 AI Model Prototype for Source Code Vulnerability Identification	부서	정보보안연구소	작성자	김건오
		최신수정일	2025.12.10	버전	1.0
		관리번호	RS-2024-00398993-2501		

- **패턴 유사성:** 일부 정상 코드가 취약점 코드와 매우 유사한 구조적 패턴을 가짐
- **문맥 정보 부족:** AST 구조만으로는 판단하기 어려운 런타임 검증 로직의 부재
- **데이터 편향:** 학습 데이터에 특정 패턴이 과도하게 포함되어 모델이 이를 과도하게 일반화

6.4.2. 개선 방향

높은 확신도의 오분류를 줄이기 위한 향후 연구 방향은 다음과 같다.

- **Adversarial Training:** 오분류된 샘플을 학습에 포함시켜 모델의 견고성 향상
- **데이터 보강:** 오분류 패턴을 포함하는 추가 학습 데이터 수집 및 생성
- **확신도 기반 필터링:** 낮은 확신도 예측에 대해서는 추가 검증 절차 적용
- **설명 가능성 분석:** 오분류 원인을 분석하여 모델의 의사결정 과정 개선

6.5. 결론

본 실험을 통해 제안된 AST 기반 GNN 모델이 실제 취약점 탐지 작업에서 높은 실용적 성능을 달성함을 확인하였다. 특히 Recall 96.39%와 Precision 97.37%, F1-Score 96.88%는, 학습된 AI 모델이 실제 취약점의 대부분을 놓치지 않으면서도 오탐을 효과적으로 억제하고 있음을 보여준다. 이는 코드의 AST 구조와 Guard, 제어 흐름 등 그래프 기반 문맥 정보를 통합적으로 학습한 결과이며, 전통적인 정적 분석 도구가 가지는 높은 False Positive와 제한적인 문맥 인식 한계를 상당 부분 완화한다는 기술적 의의를 가진다. 다만 14.35% 수준의 False Positive Rate는 여전히 개선 여지가 있으며, 향후 데이터 보강, 특징 벡터 정교화, 앙상블 및 설명 가능한 AI 기법을 결합함으로써 대규모 실무 환경에서도 안정적으로 활용 가능한 정적 취약점 탐지 플랫폼으로 확장해 나갈 수 있을 것으로 기대된다.